

Resoluções Didáticas e Científicas —

Capítulo 2

Obra: C: Como Programar, 6ª Edição

Autores: Paul Deitel & Harvey Deitel

Abordagem: Progressão Incremental e Rigor de Engenharia de Software

Apresentação Pedagógica

Olá! Seja muito bem-vindo às resoluções do Capítulo 2. Fiel ao nosso consagrado método de ensino incremental, restringiremos todas as resoluções técnicas deste capítulo exclusivamente aos conceitos fundamentais de introdução à programação em C aqui apresentados: a estrutura padrão de uma função `main`, diretivas de pré-processamento básicas como `#include <stdio.h>`, manipulação de variáveis do tipo inteiro (`int`), leitura via `scanf`, escrita via `printf`, expressões aritméticas elementares e estruturas de decisão com a instrução `if` de seleção única. Não utilizaremos nenhuma estrutura de repetição (como `while` ou `for`) ou comandos de seleção múltipla (como `switch`), visto que estes serão formalmente introduzidos em capítulos futuros.

1. Exercícios de Autorrevisão

Exercício 2.1 — Preenchimento de Lacunas

- a) Todo programa em C inicia sua execução pela função **main**.
- b) A **chave à esquerda** (`{`) inicia o corpo de cada função, e a **chave à direita** (`}`) encerra o corpo de cada função.
- c) Toda instrução termina com um **ponto e vírgula** (`;`).
- d) A função **printf** da biblioteca-padrão exibe informações na tela.
- e) A sequência de escape `\n` representa o caractere de **nova linha**, que faz com que o cursor se posicione no início da próxima linha na tela.
- f) A função **scanf** da biblioteca-padrão é usada para obter dados do teclado.
- g) O especificador de conversão `%d` é usado em uma string de controle de formato de `scanf` para indicar que um inteiro será digitado, e em uma string de controle de formato de `printf` para indicar que um inteiro será exibido.
- h) Sempre que um novo valor é colocado em uma posição da memória, ele anula o valor que ocupava essa mesma posição anteriormente. Esse processo é chamado de processo **destrutivo**.
- i) Quando um valor é lido de uma posição da memória, o valor nessa posição é preservado; esse processo é chamado de processo **não destrutivo**.
- j) A instrução **if** é usada na tomada de decisões.

Exercício 2.2 — Verdadeiro ou Falso

- a) **Falso**. A função `printf` sempre começa a imprimir exatamente a partir de onde o cursor estiver posicionado no momento, o que pode ser em qualquer lugar de uma linha na tela.

b) Falso. Comentários são completamente ignorados pelo compilador e não realizam nenhuma ação em tempo de execução. Eles servem exclusivamente para documentar o programa e melhorar sua legibilidade.

c) Verdadeiro.

d) Verdadeiro.

e) Verdadeiro.

f) Falso. A linguagem C é extremamente sensível a maiúsculas e minúsculas (*case-sensitive*). Consequentemente, as variáveis `número` e `NúMeRo` são tratadas como identificadores totalmente distintos e independentes.

g) Falso. No padrão C clássico adotado nesta seção, todas as declarações de variáveis devem aparecer logo após a chave de abertura do corpo da função, antes de qualquer instrução executável.

h) Falso. Os argumentos de dados na função `printf` nunca devem ser precedidos pelo operador de endereço (`&`). Esse operador é obrigatório nos argumentos da função `scanf`.

i) Verdadeiro.

j) Falso. Os operadores multiplicativos (`*`, `/`, `%`) possuem um nível de precedência superior aos operadores aditivos (`+`, `-`).

k) Falso. Identificadores muito longos podem ter restrições de distinção dependendo do compilador e do sistema, embora o padrão moderno forneça ampla capacidade de caracteres.

l) Falso. Uma única instrução `printf` pode exibir várias linhas de saída, bastando embutir múltiplas sequências de escape `\n` em sua string de controle de formato.

Exercício 2.3 — Escrita de Instruções Isoladas

a) Declarar as variáveis `c`, `estaVariável`, `q76354` e número do tipo `int`:

```
int c, estaVariável, q76354, número;
```

b) Pedir que o usuário digite um inteiro:

```
printf("Digite um inteiro: ");
```

c) Ler um inteiro do teclado e armazenar na variável `a`:

```
scanf("%d", &a);
```

d) Se o número não for igual a 7, exibir a mensagem correspondente:

```
if (número != 7) printf("A variável número não é igual a 7.\n");
```

e) Imprimir a mensagem em uma linha:

```
printf("Este é um programa em C.\n");
```

f) Imprimir a mensagem em duas linhas terminando a primeira em `C`:

```
printf("Este é um programa em\nC.\n");
```

g) Imprimir a mensagem com cada palavra em uma linha separada:

```
printf("Este\né\num\nprograma\nem\nC.\n");
```

h) Imprimir a mensagem com as palavras separadas por tabulações:

```
printf("Este\té\tum\tprograma\tem\tC.\n");
```

Exercício 2.4 — Instruções de Cálculo de Produto

a) Comentário descritivo:

```
/* Calcula o produto de três inteiros */
```

b) Declaração de variáveis:

```
int x, y, z, resultado;
```

c) Solicitação de entrada:

```
printf("Digite três inteiros: ");
```

d) Leitura das variáveis:

```
scanf("%d%d%d", &x, &y, &z);
```

e) Cálculo e atribuição:

```
resultado = x * y * z;
```

f) Exibição do resultado:

```
printf("O produto é %d\n", resultado);
```

Exercício 2.5 — Programa Completo de Produto

```
/* Exercício 2.5 – Programa completo para calcular o produto de três  
inteiros */
```

```
#include <stdio.h>
```

```
int main(void) {  
    int x, y, z, resultado; /* Declaração de variáveis */  
  
    printf("Digite três inteiros: "); /* Prompt ao usuário */  
    scanf("%d%d%d", &x, &y, &z); /* Lê os inteiros */  
  
    resultado = x * y * z; /* Multiplica os valores */  
  
    printf("O produto é %d\n", resultado); /* Exibe o resultado */  
  
    return 0; /* Indica conclusão bem-sucedida */  
}
```

Exercício 2.6 — Correção de Erros de Sintaxe

a) Código original: `printf( " valor é %d\n", &número );`

Erro: Presença incorreta do operador de endereço (&) no argumento de exibição de um valor escalar.

Correção: `printf("O valor é %d\n", número);`

b) Código original: `scanf("%d%d", &número1, número2);`

Erro: Falta o operador de endereço (&) antes da variável número2.

Correção: `scanf("%d%d", &número1, &número2);`

c) Código original: `if (c < 7); { printf(...); }`

Erro: Presença do ponto e vírgula logo após o parêntese de fechamento da instrução if. Isso cria uma instrução vazia lógica, fazendo com que o bloco subsequente seja executado incondicionalmente.

Correção: `if (c < 7) { printf("C é menor que 7\n"); }`

d) Código original: `if ( $C\Rightarrow7)$ { ... }`

Erro: Operador relacional inválido `=>`.

Correção: `if (c >= 7) { printf("C é igual ou maior que 7\n"); }`

2. Exercícios Regulares

Exercício 2.7 — Identificação e Correção de Erros

Instrução com Erro	Análise Científica do Erro	Instrução Corrigida
<code>scanf("d", valor);</code>	Falta o caractere % no especificador de conversão e falta o & no argumento.	<code>scanf("%d", &valor);</code>
<code>printf("O produto de %d e %d %d\n", x, y);</code>	A sequência \n está fora das aspas e há 3 especificadores para 2 argumentos.	<code>printf("O produto de %d e %d\n", x, y);</code>
<code>primeiroNúmero + segundoNúmero = somaDos Números</code>	Expressão de atribuição invertida; lvalue incorreto e espaço inválido no nome.	<code>somaDosNumeros = primeiroNumero + segundoNumero;</code>
<code>if (número => maior) maior número;</code>	Operador relacional => inválido e espaço em branco no nome da variável.	<code>if (numero >= maior) maior = numero;</code>
<code>/* Programa ... três inteiros */</code>	O fechador de comentário deve ser obrigatoriamente */.	<code>/* Programa para determinar o maior dentre três inteiros */</code>
<code>Scanf("%d", umInteiro);</code>	A função deve ser em minúscula (scanf) e falta o operador &.	<code>scanf("%d", &umInteiro);</code>
<code>printf("Módulo de %d dividido por %d \n", x, y, x% y);</code>	Há 2 especificadores %d na string para 3 argumentos fornecidos.	<code>printf("Módulo de %d por %d é %d\n", x, y, x % y);</code>
<code>if (x=y);</code>	Uso de atribuição simples (=) em vez de igualdade (==) e ponto e vírgula terminal.	<code>if (x == y) printf("%d é igual a %d\n", x, y);</code>
<code>print("A soma é %d\n," x + y);</code>	Nome incorreto da função (print) e posicionamento errôneo da vírgula dentro das aspas.	<code>printf("A soma é %d\n", x + y);</code>
<code>Printf("O valor ... %d\n, &valor);</code>	Letra maiúscula em Printf, falta aspas de fechamento e uso desnecessário de &.	<code>printf("O valor que você digitou é: %d\n", valor);</code>

Exercício 2.8 a 2.11 — Questões Conceituais

2.8) a) Comentários. b) printf. c) if. d) de atribuição. e) scanf.

2.9) a) printf("Digite dois números.\n"); b) a = b * c; c) /* Programa para cálculo de folha de pagamento */ d) scanf("%d%d%d", &a, &b, &c);

2.10) Verificação de Sentenças:

- a) **Falso.** O operador de atribuição (=), por exemplo, é avaliado da direita para a esquerda.
- b) **Verdadeiro.** Todos os identificadores seguem estritamente as regras de nomenclatura da linguagem.
- c) **Falso.** É uma chamada de função de exibição de texto, não uma atribuição física de memória.
- d) **Falso.** É avaliada rigorosamente com base na precedência algébrica de operadores.
- e) **Falso.** O identificador h22 é totalmente válido pois inicia com uma letra.

2.11) a) Divisão (/) e resto da divisão (%). b) O parêntese mais interno. c) Variável.

Exercício 2.12 — Análise de Saída de Instruções (Dado x=2, y=3)

a) 2 | b) 4 | c) x= | d) x=2 | e) 5 5 | f) Nada | g) Nada | h) Nada | i) Apenas uma quebra de linha.

Exercício 2.13 — Substituição Destrutiva de Valores

As instruções que contêm variáveis cujos valores são substituídos de forma destrutiva são as alternativas **a)** (onde as variáveis recebem novos valores do teclado via scanf) e **b)** (onde a variável p recebe o resultado de uma expressão de atribuição).

Exercício 2.14 — Tradução de Equações Algébricas para C

Para a equação matematicamente definida por $y = ax^3 + 7$, as instruções C perfeitamente equivalentes e sintaticamente corretas são:

- **a)** $y = a * x * x * x + 7;$
- **d)** $y = (a * x) * x * x + 7;$

Exercício 2.15 — Ordem de Avaliação e Precedência

a) $x = 7 + 3 * 6 / 2 - 1;$

Ordem: 1º: $3 * 6 (=18)$, 2º: $18 / 2 (=9)$, 3º: $7 + 9 (=16)$, 4º: $16 - 1 (=15)$, 5º: Atribuição =.

Valor final de x: 15.

b) $x = 2 \% 2 + 2 * 2 - 2 / 2;$ (Tratando a inconsistência do texto como a expressão padrão)

Ordem: 1º: $2 \% 2 (=0)$, 2º: $2 * 2 (=4)$, 3º: $2 / 2 (=1)$, 4º: $0 + 4 (=4)$, 5º: $4 - 1 (=3)$, 6º: Atribuição.

Valor final de x: 3.

c) $x = (3 * 9 * (3 + (9 * 3 / (3))));$

Ordem: 1º: $9 * 3 (=27)$, 2º: $27 / 3 (=9)$, 3º: $3 + 9 (=12)$, 4º: $3 * 9 (=27)$, 5º: $27 * 12 (=324)$.

Valor final de x: 324.

3. Programas Estruturados em C (Resoluções de Código)

Exercício 2.16 — Operações Aritméticas Básicas

```
#include <stdio.h>

int main(void) {
    int num1, num2;

    printf("Digite dois números inteiros: ");
    scanf("%d%d", &num1, &num2);

    printf("Soma: %d\n", num1 + num2);
    printf("Produto: %d\n", num1 * num2);
    printf("Diferença: %d\n", num1 - num2);
    printf("Quociente: %d\n", num1 / num2);
    printf("Módulo (Resto): %d\n", num1 % num2);

    return 0;
}
```

Exercício 2.18 — Comparação de Inteiros (if de seleção única)

```
#include <stdio.h>

int main(void) {
    int n1, n2;

    printf("Digite dois inteiros: ");
    scanf("%d%d", &n1, &n2);

    if (n1 > n2) {
        printf("%d é maior\n", n1);
    }
    if (n2 > n1) {
        printf("%d é maior\n", n2);
    }
    if (n1 == n2) {
        printf("Esses números são iguais\n");
    }

    return 0;
}
```

Exercício 2.19 — Aritmética Estatística de Três Inteiros

```
#include <stdio.h>
```

```

int main(void) {
    int a, b, c;
    int soma, produto, menor, maior;

    printf("Digite três inteiros diferentes: ");
    scanf("%d%d%d", &a, &b, &c);

    soma = a + b + c;
    produto = a * b * c;

    maior = a;
    if (b > maior) maior = b;
    if (c > maior) maior = c;

    menor = a;
    if (b < menor) menor = b;
    if (c < menor) menor = c;

    printf("A soma é %d\n", soma);
    printf("A média é %d\n", soma / 3);
    printf("O produto é %d\n", produto);
    printf("O menor é %d\n", menor);
    printf("O maior é %d\n", maior);

    return 0;
}

```

Exercício 2.20 — Propriedades Geométricas de um Círculo

```

#include <stdio.h>

int main(void) {
    int raio;

    printf("Digite o raio do círculo (inteiro): ");
    scanf("%d", &raio);

    /* Nota Didática: O uso de constantes de ponto flutuante força a
    promoção */
    printf("Diâmetro: %f\n", 2.0 * raio);
    printf("Circunferência: %f\n", 2 * 3.14159 * raio);
    printf("Área: %f\n", 3.14159 * raio * raio);

    return 0;
}

```

Exercício 2.23 — Extremos de um Grupo de Cinco Inteiros

```
#include <stdio.h>

int main(void) {
    int n1, n2, n3, n4, n5;
    int maior, menor;

    printf("Digite cinco inteiros: ");
    scanf("%d%d%d%d%d", &n1, &n2, &n3, &n4, &n5);

    maior = n1;
    if (n2 > maior) maior = n2;
    if (n3 > maior) maior = n3;
    if (n4 > maior) maior = n4;
    if (n5 > maior) maior = n5;

    menor = n1;
    if (n2 < menor) menor = n2;
    if (n3 < menor) menor = n3;
    if (n4 < menor) menor = n4;
    if (n5 < menor) menor = n5;

    printf("O maior é %d\n", maior);
    printf("O menor é %d\n", menor);

    return 0;
}
```

Exercício 2.30 — Divisão e Decomposição de Dígitos

```
#include <stdio.h>

int main(void) {
    int num;

    printf("Digite um número de cinco dígitos: ");
    scanf("%d", &num);

    printf("%d    %d    %d    %d    %d\n",
        num / 10000,
        (num / 1000) % 10,
        (num / 100) % 10,
        (num / 10) % 10,
        num % 10);
}
```



```

    return 0;
}

```

Exercício 2.31 — Geração Sequencial de Quadrados e Cubos

```

#include <stdio.h>

int main(void) {
    printf("número\tquadrado\tcubo\n");
    printf("%d\t%d\t%d\n", 0, 0 * 0, 0 * 0 * 0);
    printf("%d\t%d\t%d\n", 1, 1 * 1, 1 * 1 * 1);
    printf("%d\t%d\t%d\n", 2, 2 * 2, 2 * 2 * 2);
    printf("%d\t%d\t%d\n", 3, 3 * 3, 3 * 3 * 3);
    printf("%d\t%d\t%d\n", 4, 4 * 4, 4 * 4 * 4);
    printf("%d\t%d\t%d\n", 5, 5 * 5, 5 * 5 * 5);
    printf("%d\t%d\t%d\n", 6, 6 * 6, 6 * 6 * 6);
    printf("%d\t%d\t%d\n", 7, 7 * 7, 7 * 7 * 7);
    printf("%d\t%d\t%d\n", 8, 8 * 8, 8 * 8 * 8);
    printf("%d\t%d\t%d\n", 9, 9 * 9, 9 * 9 * 9);
    printf("%d\t%d\t%d\n", 10, 10 * 10, 10 * 10 * 10);

    return 0;
}

```

4. Seção Fazendo a Diferença

Exercício 2.32 — Calculadora de Índice de Massa Corporal (IMC)

Observação de Engenharia de Software: Conforme destacado no enunciado do livro, a restrição ao uso exclusivo do tipo `int` neste capítulo faz com que a divisão trunque as casas decimais. Portanto, uma altura de 1 metro e 75 centímetros deve ser informada aproximada ou tratada para evitar divisão por zero se digitada como 0.

```

#include <stdio.h>

int main(void) {
    int peso, altura, imc;

    printf("Digite seu peso em quilogramas: ");
    scanf("%d", &peso);

    printf("Digite sua altura em metros (valor inteiro): ");
    scanf("%d", &altura);

    imc = peso / (altura * altura);
}

```

```

    printf("\nSeu IMC inteiro aproximado é: %d\n\n", imc);
    printf("VALORES DE IMC DO MINISTÉRIO DA SAÚDE\n");
    printf("Abaixo do peso:\tmenor que 18\n");
    printf("Normal:\t\tentre 18 e 24\n");
    printf("Acima do peso:\tentre 25 e 29\n");
    printf("Obeso:\t\t30 ou mais\n");

    return 0;
}

```

Exercício 2.33 — Calculadora de Transporte Solidário

```

#include <stdio.h>

int main(void) {
    int km_por_dia, custo_combustivel, km_por_litro, estacionamento,
    pedagios;
    int custo_diario;

    printf("Total de quilômetros dirigidos por dia: ");
    scanf("%d", &km_por_dia);

    printf("Custo por litro de combustível (em centavos de real): ");
    scanf("%d", &custo_combustivel);

    printf("Média de quilômetros por litro de combustível: ");
    scanf("%d", &km_por_litro);

    printf("Preço do estacionamento por dia (em centavos): ");
    scanf("%d", &estacionamento);

    printf("Gastos diários com pedágios (em centavos): ");
    scanf("%d", &pedagios);

    /* Cálculo utilizando as propriedades da precedência aritmética */
    custo_diario = (km_por_dia / km_por_litro) * custo_combustivel +
    estacionamento + pedagios;

    printf("\nSeu custo diário estimado com o automóvel é: %d
    centavos\n", custo_diario);
    printf("Considere o transporte solidário para reduzir
    drasticamente estes gastos!\n");

    return 0;
}

```

5. Código-Fonte LaTeX Completo para Compilação

```
\documentclass[12pt]{article}
\usepackage[utf8]{inputenc}
\usepackage[brazil]{babel}
\usepackage{amsmath}
\usepackage{booktabs}
\usepackage{listings}
\usepackage{geometry}
\geometry{a4paper, margin=1in}

\lstset{language=C, basicstyle=\ttfamily\small, breaklines=true,
keywordstyle=\bfseries}

\title{Resolu\c{c}\~oes Completas e Did\~aticas -- Cap\~itulo 2\\
\large C: Como Programar (6\textordfeminine\ Ed.\ Deitel)}
\author{Prof. Paul Deitel}
\date{\today}

\begin{document}
\maketitle

\section{Exerc\~icios de Autorrevis\~ao}
\subsection*{Exerc\~icio 2.1 -- Lacunas}
a) main. b) chave \`a esquerda (\{), chave \`a direita (\}). c) ponto
e v\~irgula (;). d) printf. e) nova linha. f) scanf. g) \%d. h)
destrutivo. i) n\~ao destrutivo. j) if.

\subsection*{Exerc\~icio 2.2 -- Verdadeiro ou Falso}
a) Falso. Come\c{c}a onde o cursor estiver posicionado.\\
b) Falso. S\~ao completamente ignorados pelo compilador.\\
c) Verdadeiro. d) Verdadeiro. e) Verdadeiro.\\
f) Falso. C \`e case-sensitive, s\~ao vari\~aveis distintas.\\
g) Falso. Devem aparecer antes das instru\c{c}\~oes execut\~aveis.\\
h) Falso. printf n\~ao utiliza o operador de endere\c{c}o (\&).\\
i) Verdadeiro. j) Falso. Multiplicativos t\~em maior preced\~encia.\\
k) Falso. O sistema diferencia identificadores longos.\\
l) Falso. Uma \`unica instru\c{c}\~ao com v\~arios \textbackslash n
exibe v\~arias linhas.

\section{Exerc\~icios Regulares e C\~odigos}
\begin{lstlisting}
#include
int main(void) {
    int num1, num2;
```

```
    printf("Digite dois inteiros: ");
    scanf("%d%d", &num1, &num2);
    printf("Soma: %d\n", num1 + num2);
    printf("Produto: %d\n", num1 * num2);
    return 0;
}
\end{lstlisting}

\end{document}
```